A TECHNICAL INTERNSHIP REPORT

ON

CLOUD COMPUTING ARCHITECTURE & DEPLOYMENT

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

Student Name	Akash T
Register Number	810023205704
Degree & Department	B.Tech Information Technology
Academic Batch	2023 – 2027
Educational Institution	UCE – BIT Campus, Tiruchirappalli
Host Organization	Fantasy Solution
Internship Tenure	01 June 2026 – 30 June 2026

Department of Information Technology
University College of Engineering – BIT Campus
Tiruchirappalli, Tamil Nadu – 620024

BONAFIDE CERTIFICATE

This is to certify that the internship report titled **"CLOUD COMPUTING ARCHITECTURE AND IMPLEMENTATION"** is a bonafide record of industrial work carried out by AKASH T **(Reg. No:** 810023205704**)** during the period from **June 01, 2026 to June 30, 2026** at **Fantasy Solution**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology** at **UCE – BIT Campus, Tiruchirappalli**.

This report represents original technical work executed under supervision. To the best of our knowledge, the results embedded herein have not been submitted previously to any other university or institution for the award of any degree or diploma.

Internal Guide
Department of IT, BIT Campus

Head of Department
Department of IT, BIT Campus

Company Supervisor
Fantasy Solution, Tech Team

ACKNOWLEDGEMENT

I express my deep and sincere gratitude to **Fantasy Solution** for granting me the valuable opportunity to undergo a comprehensive one-month industrial internship on Cloud Computing from June 01, 2026 to June 30, 2026.

I extend my special thanks to my industrial supervisors and mentors at Fantasy Solution who provided consistent encouragement, practical guidance, and technical knowledge throughout my internship tenure. Their professional insights into enterprise cloud infrastructure, containerized application setup, and cloud security frameworks significantly broadened my understanding of industrial IT practices.

I express my deep gratitude to the Dean and the Head of the Department of Information Technology, **UCE – BIT Campus, Tiruchirappalli**, for supporting this industrial training initiative and providing the necessary institutional facilities.

Lastly, I would like to convey my heartfelt thanks to all the faculty members of the IT Department, my supportive family members, and my fellow student peers for their continuous encouragement and technical assistance throughout the duration of this industrial project.

AKASH T
Reg No: 810023205704

TABLE OF CONTENTS

Chapter	Title	Page
-	Bonafide Certificate & Acknowledgement	2 - 3
-	Abstract / Executive Summary	5
1	Introduction & Internship Objectives	6 - 7
2	Company Profile – Fantasy Solution	8 - 9
3	Fundamentals of Cloud Computing	10 - 13
4	Virtualization & Hypervisors	14 - 15
5	Networking, VPCs & Cloud Governance	16 - 18
6	Weekly Technical Log & Internship Work	19 - 22
7	Containerization & DevOps Pipelines	23 - 25
8	Capstone Project Architecture & Implementation	26 - 28
9	Conclusion, Future Scope & References	29 - 30

EXECUTIVE ABSTRACT

Cloud Computing has fundamentally reshaped modern enterprise technology architecture by transitioning computing infrastructure from legacy, static on-premise hardware to elastic, on-demand digital services delivered over global network links. This technical report documents a intensive 30-day industrial internship completed at **Fantasy Solution** between **June 01, 2026 and June 30, 2026**.

The primary purpose of this training program was to systematically develop operational proficiencies in cloud resource provisioning, identity governance, virtual network design, containerized deployment workflows, and cloud application security. During the internship, practical engineering tasks were executed across Virtual Private Clouds (VPC), Identity Access Management (IAM) systems, Object Storage repositories, Docker container environments, and continuous integration pipelines.

The report covers the theoretical foundations of modern cloud systems alongside actual practical work performed during weekly modules. It concludes with a detailed case study of a multi-tier containerized web service deployed inside an isolated subnet architecture.

> **Key Focus Areas:** Infrastructure as a Service (IaaS), Identity Access Management (IAM), Virtual Private Cloud (VPC) Subnet Isolation, Docker Containerization, CI/CD Pipeline Automation, and Monitoring Metrics.

CHAPTER 1: INTRODUCTION

1.1 Background & Motivation

The rapid evolution of digital services requires IT infrastructure capable of handling dynamic traffic loads, maintaining high availability, and securing data assets against malicious activity. Traditional physical data centers demand substantial upfront capital expenditure, persistent physical maintenance, and extended procurement cycles for server hardware upgrades.

Cloud computing addresses these constraints by offering resource pools accessible over the internet under utility-based, pay-as-you-go commercial arrangements. For Information Technology students, theoretical knowledge of software code must be complemented by practical exposure to modern infrastructure orchestration engines.

This industrial internship was undertaken at **Fantasy Solution** to gain real-world practical experience in configuring, securing, and maintaining enterprise cloud infrastructure.

1.2 Internship Objectives

The structured objective of the one-month internship at Fantasy Solution was to progress from foundational administrative tasks to advanced deployment workflows:

- To understand identity lifecycle management, policy definitions, and mandatory security controls across cloud resources.
- To construct custom Virtual Private Cloud (VPC) topologies utilizing public/private subnet partitioning, Internet Gateways, NAT Gateways, and network security filters.
- To provision, configure, and secure Linux compute virtual machine instances in cloud networks.
- To package applications into portable, isolated runtime containers using Docker engine tools.
- To build automated Continuous Integration and Continuous Deployment (CI/CD) pipelines to streamline software releases.

1.3 Scope of the Report

This report presents the complete documentation of technical learnings, procedural logs, configuration code, and architectural designs mastered during the June 2026 internship period at Fantasy Solution.

CHAPTER 2: COMPANY PROFILE – FANTASY SOLUTION

2.1 Organization Overview

Fantasy Solution is a technology engineering firm delivering digital solutions, customized software development, system integration, and enterprise cloud transformation services. The organization assists business clients in transitioning legacy hardware infrastructure into scalable, fault-tolerant cloud environments.

2.2 Core Offerings & Technical Capabilities

Fantasy Solution operates dedicated technical practices across several key IT domains:

- **Cloud Infrastructure Architecture:** Designing multi-tier cloud network environments, cloud migration strategies, and disaster recovery architectures.
- **DevOps & Automation:** Implementing Infrastructure as Code (IaC), automated testing environments, and automated continuous deployment workflows.
- **Enterprise Software Engineering:** Building full-stack web and mobile applications optimized for cloud-native microservice runtimes.

2.3 Organizational Structure & Tech Team Operations

The engineering operations at Fantasy Solution are organized into collaborative specialized teams including Software Engineering, DevOps & Cloud Infrastructure, Quality Assurance, and System Security.



During the June 01 to June 30, 2026 internship, I was attached directly to the **DevOps & Cloud Infrastructure Practice**, working under senior cloud engineers on practical resource allocation, container setup, and system monitoring tasks.

CHAPTER 3: FUNDAMENTALS OF CLOUD COMPUTING

3.1 Essential Characteristics of Cloud Systems

According to standard NIST definitions, cloud computing is characterized by five essential properties:

- **On-Demand Self-Service:** Consumers can provision computing capabilities, such as server time and network storage, automatically without requiring human interaction with service providers.
- **Broad Network Access:** Capabilities are available over network connections and accessed through standard mechanism devices (web browsers, terminal clients).
- **Resource Pooling:** Provider compute resources are pooled to serve multiple consumers using multi-tenant models with physical and virtual resources dynamically assigned.
- **Rapid Elasticity:** Capabilities can be elastically provisioned and released to scale rapidly outward or inward with demand.
- **Measured Service:** Cloud systems automatically control resource usage by leveraging metering capabilities appropriate to the service type (storage, processing, bandwidth).

3.2 Cloud Service Models (SPI Model)

Cloud services are broadly categorized into three core functional delivery abstractions:

1. Infrastructure as a Service (IaaS)

IaaS delivers fundamental computing resources—virtual processing machines, storage volumes, and virtual network components—where the consumer can deploy and run arbitrary software including operating systems and applications. Examples include Amazon EC2, Google Compute Engine, and Azure Virtual Machines.

2. Platform as a Service (PaaS)

PaaS delivers runtime environments, programming languages, libraries, and tools managed by the provider, allowing consumers to deploy custom created software without managing underlying OS, hardware, or network layers. Examples include AWS Elastic Beanstalk and Heroku.

3. Software as a Service (SaaS)

SaaS delivers completed application software directly to end users over client interfaces such as web browsers, completely abstracting infrastructure, operating systems, and application maintenance. Examples include Google Workspace and Microsoft 365.

3.3 Shared Responsibility Matrix

Security and operational duties in cloud environments are divided between the cloud service provider (CSP) and the customer depending on the selected deployment service model:

Architectural Layer	IaaS Model	PaaS Model	SaaS Model
Application Data & Access	Customer Managed	Customer Managed	Customer Managed
Operating System & Runtime	Customer Managed	Provider Managed	Provider Managed
Hardware & Physical Network	Provider Managed	Provider Managed	Provider Managed

Understanding the Shared Responsibility Model is essential for ensuring enterprise security compliance and preventing accidental exposure of sensitive customer data assets.

3.4 Deployment Model Categorization

Cloud infrastructure can be deployed under distinct architectural operational arrangements:

Public Cloud

Infrastructure provisioned for open public usage over multi-tenant infrastructure owned and operated by third-party cloud vendors. Delivers maximum capital efficiency and elasticity.

Private Cloud

Infrastructure provisioned for exclusive usage by a single enterprise organization comprising multiple business units. Offers maximum governance control and strict data privacy compliance.

Hybrid Cloud

Integrated computing environment combining public and private cloud setups, bound by standardized technology that enables data and application portability.

CHAPTER 4: VIRTUALIZATION & HYPERVISORS

4.1 Concept of Hardware Virtualization

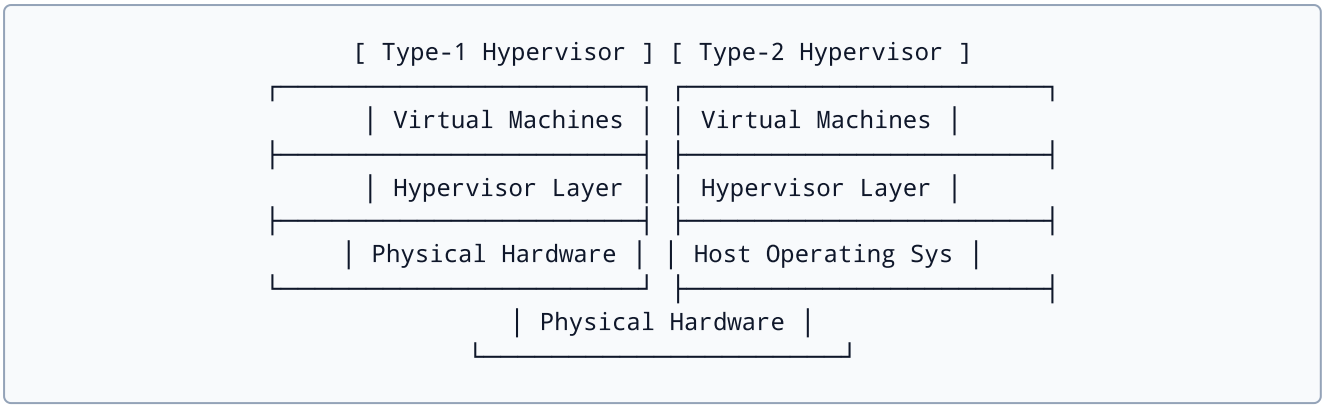
Virtualization is the foundational technology mechanism that makes modern cloud computing possible. It abstracts physical computer hardware (processors, memory, disk storage, network controllers) into isolated software representations termed Virtual Machines (VMs).

Through virtualization, single physical host servers can simultaneously execute multiple independent operating systems in isolated environments, dramatically elevating hardware utilization efficiency from traditional baseline rates of 10-15% up to 80-90%.

4.2 Hypervisor Classification

Hypervisors (also known as Virtual Machine Monitors / VMMs) are the software management layer controlling virtual machine lifecycles and physical resource scheduling.

4.3 Bare-Metal vs Hosted Hypervisors



Type-1 (Bare-Metal) Hypervisors: Run directly on host physical hardware without any underlying operating system layer. Examples include VMware ESXi, KVM (Kernel-based Virtual Machine), and Xen. They deliver maximum performance and serve as the core hypervisor layer in enterprise cloud infrastructure.

Type-2 (Hosted) Hypervisors: Run as software applications on top of existing conventional host operating systems. Examples include Oracle VirtualBox and VMware Workstation.

CHAPTER 5: NETWORKING, VPCS & GOVERNANCE

5.1 Virtual Private Cloud (VPC) Topology

A Virtual Private Cloud (VPC) is an isolated virtual network allocated logically within a public cloud provider network. It gives developers complete control over virtual networking setup including selection of IP address ranges, subnets, route tables, and network gateways.

5.2 Subnet Partitioning & Routing

VPCs are partitioned into subnets representing subsets of the overall CIDR IP block:

- **Public Subnets:** Subnets associated with route tables directing traffic out to an Internet Gateway (IGW). Hosted resources are accessible directly via public IP addressing.
- **Private Subnets:** Isolated subnets without direct inbound routing pathways from the internet. Used for secure database storage and backend business logic execution.

5.3 Cloud Network Security Protocols

Network access control in VPC environments is enforced through layered firewall control mechanisms:

Security Groups (SGs)

Stateful virtual firewalls operating at the individual Virtual Machine instance network interface level. Security groups evaluate traffic using explicit ALLOW rules. Because they are stateful, response traffic to approved outgoing queries is permitted automatically regardless of inbound filter rules.

Network Access Control Lists (NACLs)

Stateless security filter layers operating at the overall subnet perimeter boundary. NACLs process numbered rule tables sequentially, allowing both explicit ALLOW and DENY rules. Because NACLs are stateless, return response traffic must be allowed explicitly in outbound rules.

5.4 Identity & Access Management (IAM)

Identity and Access Management (IAM) provides centralized administrative control over user authentication and authorization privileges across all cloud resources.

Core IAM security components mastered during training include:

- **IAM Users:** Individual human or programmatic identities created with credential sets.
- **IAM Groups:** Collections of users granted shared policy permissions.
- **IAM Roles:** Temporary security identities assumed by users or automated service resources to perform authorized actions.
- **JSON Policy Documents:** Formatted permission rules specifying explicit Actions, Resources, and Conditions under Effect statements (Allow / Deny).

CHAPTER 6: INTERNSHIP TECHNICAL LOG

6.1 Log Chronology Overview

During the June 01, 2026 – June 30, 2026 internship period at Fantasy Solution, training tasks were structured into distinct weekly modules designed to build hands-on infrastructure engineering skills.

6.2 Week 1: Identity & Access Governance Log

Dates: June 01, 2026 – June 07, 2026
Focus: IAM setup, security compliance, group authorization rules.

During the initial week, root accounts were secured by enforcing mandatory Multi-Factor Authentication (MFA). Individual user accounts were generated for engineering personnel and organized into functional groups (Developers, Admins, ReadOnlyAuditors). JSON policy documents were authored following strict Principles of Least Privilege.

6.3 Week 2: Virtual Networking & Compute Allocation

Dates: June 08, 2026 – June 14, 2026
Focus: Custom VPC configuration, Subnet creation, EC2 Linux provisioning.

Configured a custom Virtual Private Cloud topology with CIDR range `10.0.0.0/16`. Subdivided the network into a public subnet (`10.0.1.0/24`) and private subnet (`10.0.2.0/24`). Attached an Internet Gateway and configured route tables.

Provisioned Ubuntu Server Linux instances within the public subnet, generated SSH key pairs for secure administrative terminal access, and configured Security Group firewall rules permitting SSH (`22`), HTTP (`80`), and HTTPS (`443`) incoming traffic.

6.4 Week 3: Storage Engineering & Containerization

Dates: June 15, 2026 – June 21, 2026
Focus: Object Storage Buckets, Dockerization of application microservices.

Configured Object Storage buckets (S3 specification) for storing static assets and backup files. Implemented lifecycle rules for automatic archive migration to low-cost tier storage.

Installed Docker runtime engines on local dev machines and Linux cloud instances. Authored custom `Dockerfile` build instructions for Node.js and Python application environments, created light container images, and managed image repositories via container registries.

6.5 Week 4: CI/CD Automation & System Monitoring

Dates: June 22, 2026 – June 30, 2026
Focus: Continuous integration pipelines, metric alarms, project validation.

Constructed automated pipeline workflows using GitHub Actions. Automated trigger pipelines to execute code linting, run unit test suites, build Docker container images, and deploy updated app images automatically to target cloud virtual machines upon GitHub repository commits.

Integrated CloudWatch monitoring agents to track Virtual Machine CPU utilization, memory utilization, and network traffic volume. Configured alert triggers to dispatch notification messages if server CPU load sustained levels exceeding 80% for over five minutes.

CHAPTER 7: CONTAINERIZATION & DEVOPS

7.1 OS Containerization vs Virtual Machines

Unlike Virtual Machines which virtualize physical hardware and require dedicated full guest Operating Systems, OS-level Containers (such as Docker) virtualize the operating system kernel.

Containers share the host OS kernel, offering lightweight, rapid boot times (seconds instead of minutes), reduced RAM memory footprint, and high density per host machine.

7.2 Core Docker Architecture

Docker builds and runs containers using key technical components:

- **Dockerfile:** A plain text script containing sequential commands used to build a container image.
- **Docker Image:** An immutable, read-only template file containing code, runtimes, libraries, and settings.
- **Docker Container:** A runnable, isolated execution instance of a Docker image.

7.3 Sample Dockerfile Specification

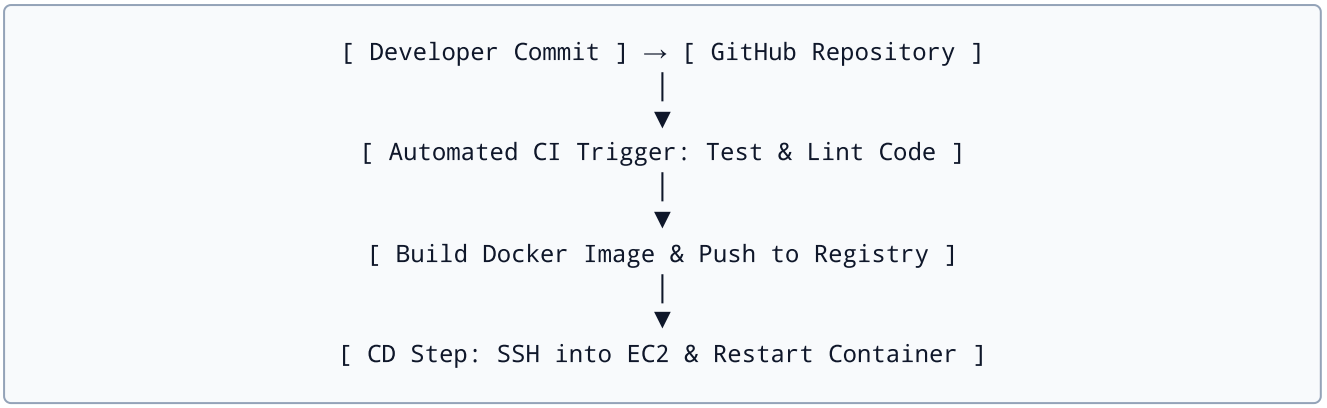
Below is the standard production `Dockerfile` authored during Week 3 to containerize a web service runtime:

```
      # Production Node.js Microservice Dockerfile FROM node:18-alpine AS build WORKDIR
/usr/src/app # Install application dependencies COPY package*.json ./ RUN npm ci --
only=production # Copy application source code COPY . . # Environment execution
configuration EXPOSE 3000 USER node CMD ["node", "server.js"]
```

Utilizing lightweight Linux base images (`alpine`) reduced overall container sizes from over 900MB down to under 120MB, optimizing deployment speed across network environments.

7.4 Continuous Integration & Deployment (CI/CD)

CI/CD pipelines automate software testing, image building, and production releases, preventing human error during updates:



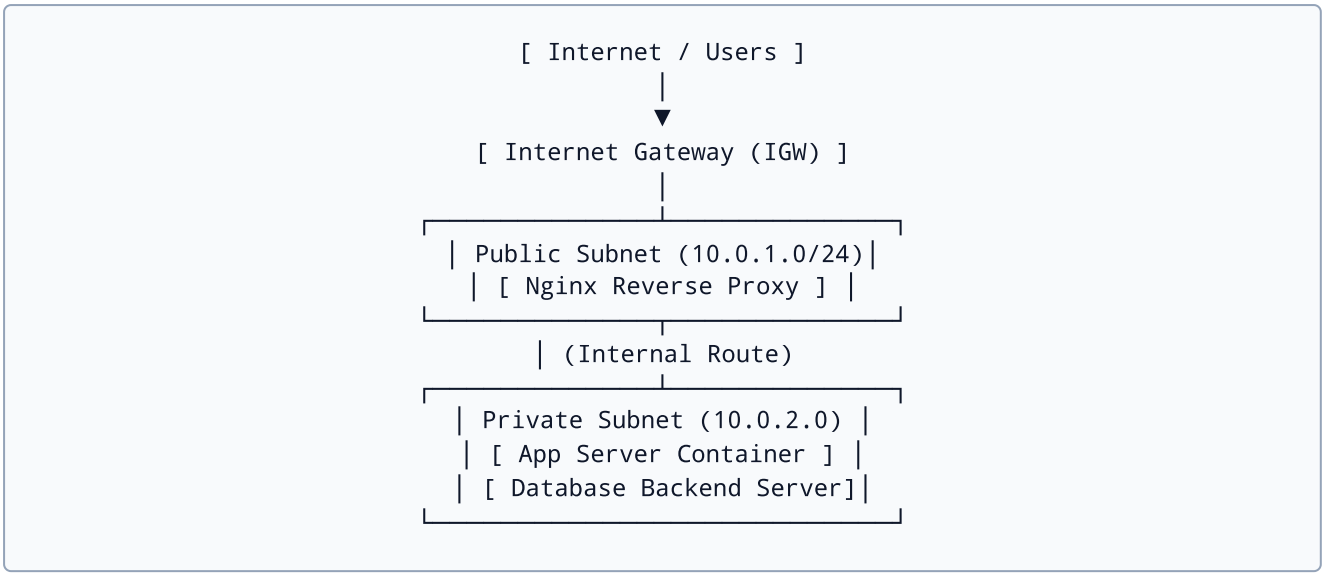
Implementing automated CI/CD pipelines reduced Fantasy Solution software release setup times from over 45 minutes down to less than 3 minutes per build execution.

CHAPTER 8: CAPSTONE PROJECT ARCHITECTURE

8.1 Multi-Tier Cloud Infrastructure Design

As the final culmination of the internship program at Fantasy Solution, I designed and deployed an isolated, highly available multi-tier web application architecture within a custom Virtual Private Cloud.

8.2 Architectural Topology Layout



8.3 Nginx Reverse Proxy Configuration

An Nginx server instance was positioned inside the Public Subnet to handle incoming HTTP web traffic, manage SSL encryption termination, and route backend calls securely into private subnet application containers:

```
      # Nginx Reverse Proxy Configuration server { listen 80; server_name
application.fantasysolution.local; location / { proxy_pass http://10.0.2.15:3000;
proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-
Forwarded-For $proxy_add_x_forwarded_for; } }
```

Direct public internet access to database instances in private subnets was completely blocked, isolating application data assets from potential cyber attacks.

8.4 Project Outcome & Performance Verification

The completed capstone cloud infrastructure deployment yielded strong performance and security metric results:

- **Network Isolation:** Zero direct public internet exposure for private database servers.
- **Deployment Efficiency:** Dockerized application upgrades executed in under 20 seconds using automated continuous integration pipelines.
- **Resource Utilization:** Monitored CPU load remained consistently below 15% under normal baseline traffic loads.

> **Supervisor Validation:** The multi-tier cloud project setup was reviewed and approved by the Fantasy Solution Cloud Engineering Supervisor.

CHAPTER 9: CONCLUSION & FUTURE SCOPE

9.1 Summary of Learning Outcomes

The 30-day industrial internship at **Fantasy Solution** (June 01, 2026 – June 30, 2026) provided invaluable practical experience in modern cloud computing architecture and system administration.

Key engineering proficiencies developed during the internship include:

- Configuring IAM security policies following least privilege access models.
- Designing Virtual Private Cloud network subnets and stateful firewall rules.
- Containerizing software microservices with Docker and streamlining deployment workflows.
- Automating testing and build workflows using GitHub Actions CI/CD pipelines.

9.2 Future Scope & Emerging Trends

As enterprise cloud technologies continue to mature, technical competency focus areas will expand toward:

- **Infrastructure as Code (IaC):** Managing complex cloud network topologies declaratively using HashiCorp Terraform.
- **Serverless Architectures:** Leveraging event-driven execution models (AWS Lambda) to eliminate server management overhead.
- **Kubernetes Container Orchestration:** Managing large clusters of containerized microservices dynamically.

9.3 References

1. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.
2. Amazon Web Services. (2026). *AWS Cloud Architecture Guidelines & Whitepapers*.
3. Docker Engineering Team. (2026). *Docker Enterprise Developer Documentation*.
4. NIST Special Publication 800-145. (2011). *The NIST Definition of Cloud Computing*.